Lecture 8. Input and Output in Haskell

Programming Paradigms, Fall 2026 — 14 Oct 2026

Nikolai Kudasov

Lab of Programming Languages and Compilers

Innopolis University



Outline

1. Where We Are
2. Programs as Values
3. Reading Input
4. A Pure Core, a Thin Shell
5. **`Functor`** and **`Monad`**

Where We Are

This Lecture

Last week: functions as values, and types with several constructors.

This Lecture

Last week: functions as values, and types with several constructors.

This week: programs that read and print, and the classes that make **`do`** work.

This Lecture



Last week: functions as values, and types with several constructors.

This week: programs that read and print, and the classes that make **`do`** work.

Four things to get out of it:

1. **Reading** a **`do`** block: the type of each name, and which actions run.
2. **Parsing** input that may be wrong, with `readMaybe` rather than `read`.
3. **Designing** a program as a pure core and a thin shell around it.
4. **Rewriting** a **`do`** block with `>>=`, for **`IO`**, **`Maybe`** and **`Either`** alike.

This Lecture

Last week: functions as values, and types with several constructors.

This week: programs that read and print, and the classes that make **`do`** work.

Four things to get out of it:

1. **Reading** a **`do`** block: the type of each name, and which actions run.
2. **Parsing** input that may be wrong, with `readMaybe` rather than `read`.
3. **Designing** a program as a pure core and a thin shell around it.
4. **Rewriting** a **`do`** block with `>>=`, for **`IO`**, **`Maybe`** and **`Either`** alike.

Last week's checks return an **`Either`**, and none of them reads or prints.
We start from two of them.

Quiz 7: Counting Functions

```
data Customer = Member | NonMember
data Country  = Russia | Kazakhstan | Belarus | Armenia
```

How many total functions have type `Country -> Customer`?
Each must return a customer for every country.

Quiz 7: Counting Functions

M = `Member`, N = `NonMember`. Each column is one function.

function	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Russia	M	M	M	M	M	M	M	M	N	N	N	N	N	N	N	N
Kazakhstan	M	M	M	M	N	N	N	N	M	M	M	M	N	N	N	N
Belarus	M	M	N	N	M	M	N	N	M	M	N	N	M	M	N	N
Armenia	M	N	M	N	M	N	M	N	M	N	M	N	M	N	M	N

Two choices for each of four inputs: $2 \times 2 \times 2 \times 2 = 16$.

Quiz 7: Counting Functions

M = `Member`, N = `NonMember`. Each column is one function.

function	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Russia	M	M	M	M	M	M	M	M	N	N	N	N	N	N	N	N
Kazakhstan	M	M	M	M	N	N	N	N	M	M	M	M	N	N	N	N
Belarus	M	M	N	N	M	M	N	N	M	M	N	N	M	M	N	N
Armenia	M	N	M	N	M	N	M	N	M	N	M	N	M	N	M	N

Two choices for each of four inputs: $2 \times 2 \times 2 \times 2 = 16$.

```
customerFor :: Country -> Customer        -- column 8
customerFor Russia = Member
customerFor _      = NonMember
```

Different code with the same outputs counts as one function.

Two Checks in a Row



```haskell
forbidden :: [String]
forbidden = ["batteries", "matches", "aerosols"]

checkContents :: String -> Either String String
checkContents c
  | c == ""             = Left "the contents must be named"
  | c `elem` forbidden = Left ("we do not carry " ++ c)
  | otherwise           = Right c
```

A second check, written as `checkWeight` was last week.

Two Checks in a Row



```haskell
checkParcelCase :: String -> Double -> Country -> Either String Parcel
checkParcelCase c w country =
  case checkContents c of
    Left err -> Left err
    Right c' ->
      case checkWeight w of
        Left err -> Left err
        Right w' -> Right (Parcel c' w' country)
```

A parcel needs both checks.

Each check is a `case`, and the next check goes inside its `Right`.

Two Checks in a Row

```
checkParcelCase :: String -> Double -> Country -> Either String Parcel
checkParcelCase c w country =
  case checkContents c of
    Left err -> Left err
    Right c' ->
      case checkWeight w of
        Left err -> Left err
        Right w' -> Right (Parcel c' w' country)
```

A parcel needs both checks.
Each check is a `case`, and the next check goes inside its `Right`.

What does a third check do to this code?

Two Checks in a Row



```haskell
checkParcelCase :: String -> Double -> Country -> Either String Parcel
checkParcelCase c w country =
  case checkContents c of
    Left err -> Left err
    Right c' ->
      case checkWeight w of
        Left err -> Left err
        Right w' -> Right (Parcel c' w' country)
```

A parcel needs both checks.
Each check is a `case`, and the next check goes inside its `Right`.

What does a third check do to this code?
It goes one step further to the right, with one more `Left err -> Left err`.

do Flattens the Staircase



```haskell
checkParcel :: String -> Double -> Country -> Either String Parcel
checkParcel c w country = do
  c' <- checkContents c
  w' <- checkWeight w
  Right (Parcel c' w' country)
```

do Flattens the Staircase



```haskell
checkParcel :: String -> Double -> Country -> Either String Parcel
checkParcel c w country = do
  c' <- checkContents c
  w' <- checkWeight w
  Right (Parcel c' w' country)
```

```
ghci>  checkParcel "books" 5400 Armenia
Right (Parcel {contents = "books", weight = 5400.0, destination = Armenia})
ghci>  checkParcel "matches" 50 Russia
Left "we do not carry matches"
```

`<-` names the value inside a `Right`, and a `Left` ends the block.

do Flattens the Staircase



```haskell
checkParcel :: String -> Double -> Country -> Either String Parcel
checkParcel c w country = do
  c' <- checkContents c
  w' <- checkWeight w
  Right (Parcel c' w' country)
```

```
ghci>  checkParcel "books" 5400 Armenia
Right (Parcel {contents = "books", weight = 5400.0, destination = Armenia})
ghci>  checkParcel "matches" 50 Russia
Left "we do not carry matches"
```

`<-` names the value inside a `Right`, and a `Left` ends the block.

The last section shows where the cases `Left err -> Left err` went.

do Flattens the Staircase

```haskell
checkParcel :: String -> Double -> Country -> Either String Parcel
checkParcel c w country = do
  c' <- checkContents c
  w' <- checkWeight w
  Right (Parcel c' w' country)
```

```
ghci>  checkParcel "books" 5400 Armenia
Right (Parcel {contents = "books", weight = 5400.0, destination = Armenia})
ghci>  checkParcel "matches" 50 Russia
Left "we do not carry matches"
```

`<-` names the value inside a `Right`, and a `Left` ends the block.

The last section shows where the cases `Left err -> Left err` went.

Note that no function so far reads input or prints.
Today we write such functions.

Programs as Values

An Action is a Value



```haskell
hello :: IO ()
hello = putStrLn "the depot is open"
```

```
ghci> :type putStrLn
putStrLn :: String -> IO ()
```

An Action is a Value



```haskell
hello :: IO ()
hello = putStrLn "the depot is open"
```

```
ghci> :type putStrLn
putStrLn :: String -> IO ()
```

A value of type `IO a` is an **action**: a program that may read and print, then gives an `a`.

`()` has one value, also written `()`. An `IO ()` gives nothing of interest.

An Action is a Value



```haskell
hello :: IO ()
hello = putStrLn "the depot is open"
```

```
ghci> :type putStrLn
putStrLn :: String -> IO ()
ghci> hello
the depot is open
```

A value of type `IO a` is an **action**: a program that may read and print, then gives an `a`.
`()` has one value, also written `()`. An `IO ()` gives nothing of interest.

Defining `hello` prints nothing, and running it at the GHCi prompt prints a line.
A compiled program runs one action, `main :: IO ()`.

An Action is a Value



```haskell
hello :: IO ()
hello = putStrLn "the depot is open"
```

```
ghci> :type putStrLn
putStrLn :: String -> IO ()
ghci> hello
the depot is open
```

A value of type `IO a` is an **action**: a program that may read and print, then gives an `a`.
`()` has one value, also written `()`. An `IO ()` gives nothing of interest.

Defining `hello` prints nothing, and running it at the GHCi prompt prints a line.
A compiled program runs one action, `main :: IO ()`.

In Racket, `(displayln s)` printed as soon as it was evaluated.
In Haskell, the type shows the effect, and the action prints when run.

Reading a Line

```haskell
echo :: IO ()
echo = do
  line <- getLine
  putStrLn ("you said: " ++ line)
```

```
ghci> :type getLine
getLine :: IO String
```

Reading a Line

```haskell
echo :: IO ()
echo = do
  line <- getLine
  putStrLn ("you said: " ++ line)
```

```
ghci> :type getLine
getLine :: IO String
ghci> echo
books for Armenia  (typed)
you said: books for Armenia
```

A **do** block runs its actions in order, one per line.
The whole block is one action, here an **IO ()**.

Reading a Line



```haskell
echo :: IO ()
echo = do
  line <- getLine
  putStrLn ("you said: " ++ line)
```

```
ghci> :type getLine
getLine :: IO String
ghci> echo
books for Armenia  (typed)
you said: books for Armenia
```

A **do** block runs its actions in order, one per line.
The whole block is one action, here an **IO ()**.

`line <- getLine` runs `getLine` and names the line it reads.
What is the type of `line`?

Reading a Line

```haskell
echo :: IO ()
echo = do
  line <- getLine
  putStrLn ("you said: " ++ line)
```

```
ghci> :type getLine
getLine :: IO String
ghci> echo
books for Armenia  (typed)
you said: books for Armenia
```

A `do` block runs its actions in order, one per line.
The whole block is one action, here an `IO ()`.

`line <- getLine` runs `getLine` and names the line it reads.
What is the type of `line`?

`String`. `getLine` is an `IO String`, and `line` is the string it gives.

Question: Does It Compile?

```haskell
echoWrong :: IO ()
echoWrong = putStrLn ("you said: " ++ getLine)
```

Question

Is `"you said: " ++ getLine` a string?

Question: Does It Compile?



```haskell
echoWrong :: IO ()
echoWrong = putStrLn ("you said: " ++ getLine)
```

Question

Is `"you said: " ++ getLine` a string?

```
Parcels.hs:4:39: error: [GHC-83865]
    • Couldn't match expected type: [Char]
                  with actual type: IO String
    • In the second argument of ‘(++)’, namely ‘getLine’
```

No. `++` needs a string, and `getLine` is an action that gives one when it runs. GHC writes the alias **`String`** as its definition, `[`**`Char`**`]`.

Question: Does It Compile?

```haskell
echoWrong :: IO ()
echoWrong = putStrLn ("you said: " ++ getLine)
```

Question

Is `"you said: " ++ getLine` a string?

```
Parcels.hs:4:39: error: [GHC-83865]
    • Couldn't match expected type: [Char]
                  with actual type: IO String
    • In the second argument of ‘(++)’, namely ‘getLine’
```

No. `++` needs a string, and `getLine` is an action that gives one when it runs. GHC writes the alias **`String`** as its definition, `[Char]`.

So the string exists only inside another action, after `<-`.

A do Block in IO

```
label :: IO ()
label = do
  c       <- getLine
  country <- getLine
  let text = c ++ " to " ++ country
  putStrLn ("label: " ++ text)
  putStrLn "next parcel, please"
```

A do Block in IO

```haskell
label :: IO ()
label = do
  c       <- getLine
  country <- getLine
  let text = c ++ " to " ++ country
  putStrLn ("label: " ++ text)
  putStrLn "next parcel, please"
```

What are the types of `c`, of `text`, and of the last line?

A do Block in IO

```
label :: IO ()
label = do
  c       <- getLine
  country <- getLine
  let text = c ++ " to " ++ country
  putStrLn ("label: " ++ text)
  putStrLn "next parcel, please"
```

What are the types of `c`, of `text`, and of the last line?

`String`, **`String`** and **`IO ()`**. The last line gives its type to the whole block.

A do Block in IO

```
label :: IO ()
label = do
  c       <- getLine
  country <- getLine
  let text = c ++ " to " ++ country
  putStrLn ("label: " ++ text)
  putStrLn "next parcel, please"
```

What are the types of `c`, of `text`, and of the last line?
`String`, **`String`** and **`IO ()`**. The last line gives its type to the whole block.

`<-` names the result of an action. **`let`** names a pure value.
Inside a block, a **`let`** has no **`in`**: its names are known in the lines below it.

A do Block in IO

```
label :: IO ()
label = do
  c       <- getLine
  country <- getLine
  let text = c ++ " to " ++ country
  putStrLn ("label: " ++ text)
  putStrLn "next parcel, please"
```

What are the types of `c`, of `text`, and of the last line?
`String`, **`String`** and **`IO ()`**. The last line gives its type to the whole block.

`<-` names the result of an action. **`let`** names a pure value.
Inside a block, a **`let`** has no **`in`**: its names are known in the lines below it.

A line without either runs its action, and nothing is named.

A Result Thrown Away

```haskell
weighIn :: IO ()
weighIn = do
  putStrLn "put the parcel on the scale and press Enter"
  getLine
  putStrLn "weighing"
```

A Result Thrown Away



```haskell
weighIn :: IO ()
weighIn = do
  putStrLn "put the parcel on the scale and press Enter"
  getLine
  putStrLn "weighing"
```

The program waits for Enter and ignores the line.

With `-Wall`, GHC asks about it:

```
Parcels.hs:6:3: warning: [GHC-81995] [-Wunused-do-bind]
    A do-notation statement discarded a result of type ‘String’
    Suggested fix: Suppress this warning by saying ‘_ <- getLine’
```

A Result Thrown Away

```haskell
weighIn :: IO ()
weighIn = do
  putStrLn "put the parcel on the scale and press Enter"
  getLine
  putStrLn "weighing"
```

The program waits for Enter and ignores the line.
With `-Wall`, GHC asks about it:

```
Parcels.hs:6:3: warning: [GHC-81995] [-Wunused-do-bind]
    A do-notation statement discarded a result of type ‘String’
    Suggested fix: Suppress this warning by saying ‘_ <- getLine’
```

`_ <- getLine` says that we drop the line on purpose.
For `putStrLn`, the result is `()`, and GHC does not ask.

Question: What Does It Print?

```haskell
calls :: IO ()
calls = do
  let call  = putStrLn "next, please"
      queue = [call, call, call]
  call
  call
  putStrLn ("still waiting: " ++ show (length queue))
```

Question

How often is “next, please” printed?

Question: What Does It Print?

```haskell
calls :: IO ()
calls = do
  let call  = putStrLn "next, please"
      queue = [call, call, call]
  call
  call
  putStrLn ("still waiting: " ++ show (length queue))
```

Question

How often is “next, please” printed?

```
ghci> calls
next, please
next, please
still waiting: 3
```

Question: What Does It Print?

```haskell
calls :: IO ()
calls = do
  let call  = putStrLn "next, please"
      queue = [call, call, call]
  call
  call
  putStrLn ("still waiting: " ++ show (length queue))
```

Question

How often is “next, please” printed?

```
ghci> calls
next, please
next, please
still waiting: 3
```

Each of the two executions of `call` prints once.

Question: What Does It Print?

```haskell
calls :: IO ()
calls = do
  let call  = putStrLn "next, please"
      queue = [call, call, call]
  call
  call
  putStrLn ("still waiting: " ++ show (length queue))
```

Question

How often is “next, please” printed?

```
ghci> calls
next, please
next, please
still waiting: 3
```

Each of the two executions of `call` prints once.
Neither **`let`** nor `length` runs an action.

`pure`: An Action That Does Nothing



```haskell
warnIfHeavy :: Parcel -> IO ()
warnIfHeavy p =
  if weight p > 20000
    then putStrLn ("two people to carry the " ++ contents p)
    else pure ()
```

`pure`: An Action That Does Nothing

```haskell
warnIfHeavy :: Parcel -> IO ()
warnIfHeavy p =
  if weight p > 20000
    then putStrLn ("two people to carry the " ++ contents p)
    else pure ()
```

Both branches of an `if` are actions.

`pure ()` is the one that does nothing.

`pure`: An Action That Does Nothing



```haskell
warnIfHeavy :: Parcel -> IO ()
warnIfHeavy p =
  if weight p > 20000
    then putStrLn ("two people to carry the " ++ contents p)
    else pure ()
```

Both branches of an `if` are actions.

`pure ()` is the one that does nothing.

`pure v` reads and prints nothing, and gives `v`:

```haskell
pure :: a -> IO a       -- in IO; GHCi prints a more general type
```

`pure` Does Not End the Block



```haskell
closeDepot :: IO ()
closeDepot = do
  putStrLn "closing the depot"
  pure ()
  putStrLn "the depot is closed"
```

`pure` Does Not End the Block



What does `closeDepot` print?

```haskell
closeDepot :: IO ()
closeDepot = do
  putStrLn "closing the depot"
  pure ()
  putStrLn "the depot is closed"
```

`pure` Does Not End the Block



```haskell
closeDepot :: IO ()
closeDepot = do
  putStrLn "closing the depot"
  pure ()
  putStrLn "the depot is closed"
```

What does `closeDepot` print?

```
ghci>  closeDepot
closing the depot
the depot is closed
```

Both lines, since `pure` does not end the block, unlike Python's `return`.
A block ends at its last line, and only there.

`pure` Does Not End the Block

```
closeDepot :: IO ()
closeDepot = do
  putStrLn "closing the depot"
  pure ()
  putStrLn "the depot is closed"
```

What does `closeDepot` print?

```
ghci> closeDepot
closing the depot
the depot is closed
```

Both lines, since `pure` does not end the block, unlike Python's `return`.
A block ends at its last line, and only there.

Older code, and *Learn You a Haskell*, write `return`.
For the monads of the last section, it is the same function as `pure`.

Reading Input

read and readMaybe



```haskell
import Text.Read (readMaybe)
read      :: Read a => String -> a
readMaybe :: Read a => String -> Maybe a
```

```
ghci> read "5400" :: Double
5400.0
```

read and readMaybe



```
import Text.Read (readMaybe)
read      :: Read a => String -> a
readMaybe :: Read a => String -> Maybe a
```

```
ghci>  read "5400" :: Double
5400.0
```

The class `Read` provides parsing, and `:: Double` says which type we want.

read and readMaybe

```
import Text.Read (readMaybe)
read      :: Read a => String -> a
readMaybe :: Read a => String -> Maybe a
```

```
ghci>  read "5400" :: Double
5400.0
ghci>  read "5.4kg" :: Double
*** Exception: Prelude.read: no parse
```

The class `Read` provides parsing, and `:: Double` says which type we want.

On a bad string, `read` stops the whole program.

read and readMaybe

```haskell
import Text.Read (readMaybe)
read      :: Read a => String -> a
readMaybe :: Read a => String -> Maybe a
```

```
ghci> read "5400" :: Double
5400.0
ghci> read "5.4kg" :: Double
*** Exception: Prelude.read: no parse
ghci> readMaybe "5.4kg" :: Maybe Double
Nothing
```

The class `Read` provides parsing, and `:: Double` says which type we want.

On a bad string, `read` stops the whole program.
`readMaybe` gives `Nothing`, and the program decides what to do.

read and readMaybe



```haskell
import Text.Read (readMaybe)
read      :: Read a => String -> a
readMaybe :: Read a => String -> Maybe a
```

```
ghci> read "5400" :: Double
5400.0
ghci> read "5.4kg" :: Double
*** Exception: Prelude.read: no parse
ghci> readMaybe "5.4kg" :: Maybe Double
Nothing
```

The class `Read` provides parsing, and `:: Double` says which type we want.

On a bad string, `read` stops the whole program.
`readMaybe` gives `Nothing`, and the program decides what to do.

Input comes from people, and people mistype. So we use `readMaybe`.

Reading Our Own Types



```haskell
data Country = Russia | Kazakhstan | Belarus | Armenia
  deriving (Show, Read, Eq, Ord)
```

```
ghci>  readMaybe "Armenia" :: Maybe Country
Just Armenia
```

A derived **Read** reads a value in the form that **Show** writes.

Reading Our Own Types



```haskell
data Country = Russia | Kazakhstan | Belarus | Armenia
  deriving (Show, Read, Eq, Ord)
```

```
ghci>  readMaybe "Armenia" :: Maybe Country
Just Armenia
ghci>  readMaybe "armenia" :: Maybe Country
```

A derived **`Read`** reads a value in the form that **`Show`** writes.

Reading Our Own Types



```haskell
data Country = Russia | Kazakhstan | Belarus | Armenia
  deriving (Show, Read, Eq, Ord)
```

```
ghci>  readMaybe "Armenia" :: Maybe Country
Just Armenia
ghci>  readMaybe "armenia" :: Maybe Country
Nothing
```

A derived **`Read`** reads a value in the form that **`Show`** writes.

So a clerk who types `armenia` gets no country.
The type **`Maybe Country`** makes us decide what to tell the clerk.

Question: One Block, Two Types?

```haskell
parseWrong :: String -> String -> String -> Either String Parcel
parseWrong c ws cs = do
  w       <- readMaybe ws
  country <- readMaybe cs
  checkParcel c w country
```

Does it compile?

Question: One Block, Two Types?



```haskell
parseWrong :: String -> String -> String -> Either String Parcel
parseWrong c ws cs = do
  w       <- readMaybe ws
  country <- readMaybe cs
  checkParcel c w country
```

Does it compile?

```
Parcels.hs:14:14: error: [GHC-83865]
    • Couldn't match type ‘Maybe’ with ‘Either String’
      Expected: Either String Double
        Actual: Maybe Double
```

`readMaybe` gives **`Maybe`** where this block needs **`Either String`**.

Question: One Block, Two Types?



```haskell
parseWrong :: String -> String -> String -> Either String Parcel
parseWrong c ws cs = do
  w       <- readMaybe ws
  country <- readMaybe cs
  checkParcel c w country
```

Does it compile?

```
Parcels.hs:14:14: error: [GHC-83865]
    • Couldn't match type ‘Maybe’ with ‘Either String’
      Expected: Either String Double
        Actual: Maybe Double
```

`readMaybe` gives **`Maybe`** where this block needs **`Either String`**.

Besides, a **`Nothing`** has no reason to give the clerk.

Task: From `Maybe` to `Either`

```
readField :: Read a => String -> String -> Either String a
```

```
ghci>  readField "weight" "5400" :: Either String Double
Right 5400.0
ghci>  readField "weight" "ninety" :: Either String Double
Left "cannot read the weight: ninety"
```

Two minutes, on paper

Write `readField`.
The first argument names the field.

Hint

A **`case`** on `readMaybe s`.

Task: From `Maybe` to `Either`, Solved



```haskell
readField :: Read a => String -> String -> Either String a
readField what s =
  case readMaybe s of
    Nothing -> Left ("cannot read the " ++ what ++ ": " ++ s)
    Just x  -> Right x
```

Task: From `Maybe` to `Either`, Solved



```haskell
readField :: Read a => String -> String -> Either String a
readField what s =
  case readMaybe s of
    Nothing -> Left ("cannot read the " ++ what ++ ": " ++ s)
    Just x  -> Right x
```

The constraint `Read a` comes from `readMaybe`, as `Ord a` came from `>` last week.

Task: From `Maybe` to `Either`, Solved



```haskell
readField :: Read a => String -> String -> Either String a
readField what s =
  case readMaybe s of
    Nothing -> Left ("cannot read the " ++ what ++ ": " ++ s)
    Just x  -> Right x
```

The constraint `Read a` comes from `readMaybe`, as `Ord a` came from `>` last week.

Both reads now fit in a block in `Either String`:

```haskell
w       <- readField "weight" ws
country <- readField "country" cs
checkParcel c w country
```

Parsing a Line



```haskell
parseParcel :: [String] -> Either String Parcel
parseParcel [c, ws, cs] = do
  w       <- readField "weight" ws
  country <- readField "country" cs
  checkParcel c w country
parseParcel _ = Left "expected the contents, the weight and the country"
```

```
ghci>  parseParcel (words "books 5400 Armenia")
Right (Parcel {contents = "books", weight = 5400.0, destination = Armenia})
```

Parsing a Line



```haskell
parseParcel :: [String] -> Either String Parcel
parseParcel [c, ws, cs] = do
  w       <- readField "weight" ws
  country <- readField "country" cs
  checkParcel c w country
parseParcel _ = Left "expected the contents, the weight and the country"
```

```
ghci> parseParcel (words "books 5400 Armenia")
Right (Parcel {contents = "books", weight = 5400.0, destination = Armenia})
```

How does `readField` know which type to read?

Parsing a Line



```haskell
parseParcel :: [String] -> Either String Parcel
parseParcel [c, ws, cs] = do
  w       <- readField "weight" ws
  country <- readField "country" cs
  checkParcel c w country
parseParcel _ = Left "expected the contents, the weight and the country"
```

```
ghci>  parseParcel (words "books 5400 Armenia")
Right (Parcel {contents = "books", weight = 5400.0, destination = Armenia})
ghci>  parseParcel (words "winter boots 1800 Belarus")
Left "expected the contents, the weight and the country"
```

How does `readField` know which type to read?

From `checkParcel`, which takes a **`Double`** and a **`Country`**.

Parsing a Line



```haskell
parseParcel :: [String] -> Either String Parcel
parseParcel [c, ws, cs] = do
  w       <- readField "weight" ws
  country <- readField "country" cs
  checkParcel c w country
parseParcel _ = Left "expected the contents, the weight and the country"
```

```
ghci>  parseParcel (words "books 5400 Armenia")
Right (Parcel {contents = "books", weight = 5400.0, destination = Armenia})
ghci>  parseParcel (words "winter boots 1800 Belarus")
Left "expected the contents, the weight and the country"
```

How does `readField` know which type to read?

From `checkParcel`, which takes a **`Double`** and a **`Country`**.

`words` splits at spaces, so contents of two words do not fit this format.

5 min break



A Pure Core, a Thin Shell

The Clerk’s Commands

A clerk types one command per line:

```
add books 5400 Armenia
list
total
quit
```

```haskell
data Command
  = Accept Parcel
  | ListAll
  | Total
  | Close
  deriving Show
```

```haskell
parseCommand :: String -> Either String Command
parseCommand line =
  case words line of
    ("add" : fields) -> do
      p <- parseParcel fields
      Right (Accept p)
    ["list"]  -> Right ListAll
    ["total"] -> Right Total
    ["quit"]  -> Right Close
    _         -> Left ("unknown command: " ++ line)
```

The Clerk's Commands

A clerk types one command per line:

```
add books 5400 Armenia
list
total
quit
```

```haskell
data Command
  = Accept Parcel
  | ListAll
  | Total
  | Close
  deriving Show
```

```haskell
parseCommand :: String -> Either String Command
parseCommand line =
  case words line of
    ("add" : fields) -> do
      p <- parseParcel fields
      Right (Accept p)
    ["list"]  -> Right ListAll
    ["total"] -> Right Total
    ["quit"]  -> Right Close
    _         -> Left ("unknown command: " ++ line)
```

A line becomes a value of our own type.
The rest of the program never looks at the string again.

One Step, Pure



```haskell
-- describe :: Parcel -> String, summary :: [Parcel] -> String
step :: Command -> [Parcel] -> ([String], Maybe [Parcel])
step (Accept p) ps = (["accepted: " ++ describe p], Just (ps ++ [p]))
step ListAll []    = (["no parcels yet"], Just [])
step ListAll ps    = (zipWith numbered [1 ..] (map describe ps), Just ps)
step Total ps      = ([summary ps], Just ps)
step Close ps      = (["closing with " ++ summary ps], Nothing)
```

One Step, Pure



```haskell
-- describe :: Parcel -> String, summary :: [Parcel] -> String
step :: Command -> [Parcel] -> ([String], Maybe [Parcel])
step (Accept p) ps = (["accepted: " ++ describe p], Just (ps ++ [p]))
step ListAll []    = (["no parcels yet"], Just [])
step ListAll ps    = (zipWith numbered [1 ..] (map describe ps), Just ps)
step Total ps      = ([summary ps], Just ps)
step Close ps      = (["closing with " ++ summary ps], Nothing)
```

A step gives the lines to print, and the parcels for the next step or `Nothing`.

One Step, Pure



```haskell
-- describe :: Parcel -> String, summary :: [Parcel] -> String
step :: Command -> [Parcel] -> ([String], Maybe [Parcel])
step (Accept p) ps = (["accepted: " ++ describe p], Just (ps ++ [p]))
step ListAll []    = (["no parcels yet"], Just [])
step ListAll ps    = (zipWith numbered [1 ..] (map describe ps), Just ps)
step Total ps      = ([summary ps], Just ps)
step Close ps      = (["closing with " ++ summary ps], Nothing)
```

A step gives the lines to print, and the parcels for the next step or **`Nothing`**.

```
ghci>  step ListAll []
(["no parcels yet"],Just [])
ghci>  step Close [books]
(["closing with parcels: 1, weight: 5400.0 g, postage: 900.0"],Nothing)
```

`step` reads and prints nothing, so we test it at the prompt.

One Step, Pure



```haskell
-- describe :: Parcel -> String, summary :: [Parcel] -> String
step :: Command -> [Parcel] -> ([String], Maybe [Parcel])
step (Accept p) ps = (["accepted: " ++ describe p], Just (ps ++ [p]))
step ListAll []    = (["no parcels yet"], Just [])
step ListAll ps    = (zipWith numbered [1 ..] (map describe ps), Just ps)
step Total ps      = ([summary ps], Just ps)
step Close ps      = (["closing with " ++ summary ps], Nothing)
```

A step gives the lines to print, and the parcels for the next step or **`Nothing`**.

```
ghci>  step ListAll []
(["no parcels yet"],Just [])
ghci>  step Close [books]
(["closing with parcels: 1, weight: 5400.0 g, postage: 900.0"],Nothing)
```

`step` reads and prints nothing, so we test it at the prompt.

Without the **`Maybe`**, `step c` has the shape `s -> (a, s)`.

Question: Why Not `map`?



A step gives a list of replies. Does this print them?

```
printAll :: [String] -> IO ()
printAll replies = map putStrLn replies
```

Question: Why Not `map`?

A step gives a list of replies. Does this print them?

```
printAll :: [String] -> IO ()
printAll replies = map putStrLn replies
```

```
Parcels.hs:4:20: error: [GHC-83865]
    • Couldn't match expected type: IO ()
                  with actual type: [IO ()]
```

No. `map` builds a list of actions, as `queue` did, and runs none of them.

Question: Why Not `map`?



A step gives a list of replies. Does this print them?

```
printAll :: [String] -> IO ()
printAll replies = map putStrLn replies
```

```
Parcels.hs:4:20: error: [GHC-83865]
    • Couldn't match expected type: IO ()
                  with actual type: [IO ()]
```

No. `map` builds a list of actions, as `queue` did, and runs none of them.

`mapM_` runs an action for each element, in order:

```
mapM_ :: (a -> IO b) -> [a] -> IO ()      -- for a list and IO
```

```
ghci> mapM_ putStrLn ["1. books", "2. kettle"]
1. books
2. kettle
```

A Loop is a Recursion



```haskell
clerk :: [Parcel] -> IO ()
clerk ps = do
  line <- getLine
  case parseCommand line of
    Left err -> do
      putStrLn err
      clerk ps
    Right command -> do
      let (replies, next) = step command ps
      mapM_ putStrLn replies
      case next of
        Nothing  -> pure ()
        Just ps' -> clerk ps'
```

A Loop is a Recursion

```haskell
clerk :: [Parcel] -> IO ()
clerk ps = do
  line <- getLine
  case parseCommand line of
    Left err -> do
      putStrLn err
      clerk ps
    Right command -> do
      let (replies, next) = step command ps
      mapM_ putStrLn replies
      case next of
        Nothing  -> pure ()
        Just ps' -> clerk ps'
```

`clerk ps` reads a command, prints the replies, and calls itself with the next parcels.

A Loop is a Recursion

```haskell
clerk :: [Parcel] -> IO ()
clerk ps = do
  line <- getLine
  case parseCommand line of
    Left err -> do
      putStrLn err
      clerk ps
    Right command -> do
      let (replies, next) = step command ps
      mapM_ putStrLn replies
      case next of
        Nothing  -> pure ()
        Just ps' -> clerk ps'
```

`clerk ps` reads a command, prints the replies, and calls itself with the next parcels.

The loop ends when `step` gives `Nothing`, since that branch does not call `clerk`.

A Loop is a Recursion

```haskell
clerk :: [Parcel] -> IO ()
clerk ps = do
  line <- getLine
  case parseCommand line of
    Left err -> do
      putStrLn err
      clerk ps
    Right command -> do
      let (replies, next) = step command ps
      mapM_ putStrLn replies
      case next of
        Nothing  -> pure ()
        Just ps' -> clerk ps'
```

`clerk ps` reads a command, prints the replies, and calls itself with the next parcels.

The loop ends when `step` gives `Nothing`, since that branch does not call `clerk`.

No list changes in place. The parcels are an argument, like an accumulator.

A Loop is a Recursion

```haskell
clerk :: [Parcel] -> IO ()
clerk ps = do
  line <- getLine
  case parseCommand line of
    Left err -> do
      putStrLn err
      clerk ps
    Right command -> do
      let (replies, next) = step command ps
      mapM_ putStrLn replies
      case next of
        Nothing  -> pure ()
        Just ps' -> clerk ps'
```

`clerk ps` reads a command, prints the replies, and calls itself with the next parcels.

The loop ends when `step` gives `Nothing`, since that branch does not call `clerk`.

No list changes in place. The parcels are an argument, like an accumulator.

`step` makes every decision about the parcels, and `clerk` only reads and prints.

The Clerk at Work



lecture-8-session.txt

```
$ runghc lecture-8.hs < lecture-8-session.txt
```

```
list
add books 5400 Armenia
add kettle 1200 Kazakhstan
add matches 50 Russia
add piano 250000 Belarus
add socks ninety Russia
add socks 90 russia
add socks 90 Russia
list
total
send everything
quit
```

The Clerk at Work



lecture-8-session.txt

```
list
add books 5400 Armenia
add kettle 1200 Kazakhstan
add matches 50 Russia
add piano 250000 Belarus
add socks ninety Russia
add socks 90 russia
add socks 90 Russia
list
total
send everything
quit
```

```
$ runghc lecture-8.hs < lecture-8-session.txt
```

```
depot open: add, list, total, quit
no parcels yet
accepted: books, 5400.0 g to Armenia
accepted: kettle, 1200.0 g to Kazakhstan
we do not carry matches
too heavy for a courier
cannot read the weight: ninety
cannot read the country: russia
accepted: socks, 90.0 g to Russia
1. books, 5400.0 g to Armenia
2. kettle, 1200.0 g to Kazakhstan
3. socks, 90.0 g to Russia
parcels: 3, weight: 6690.0 g, postage: 1550.0
unknown command: send everything
closing with parcels: 3, weight: 6690.0 g, postage: 1550.0
```

The Clerk at Work



lecture-8-session.txt

```
list
add books 5400 Armenia
add kettle 1200 Kazakhstan
add matches 50 Russia
add piano 250000 Belarus
add socks ninety Russia
add socks 90 russia
add socks 90 Russia
list
total
send everything
quit
```

`runghc` runs a module without building a program file.

With <, `getLine` reads the file, one line per call.

```
$ runghc lecture-8.hs < lecture-8-session.txt
depot open: add, list, total, quit
no parcels yet
accepted: books, 5400.0 g to Armenia
accepted: kettle, 1200.0 g to Kazakhstan
we do not carry matches
too heavy for a courier
cannot read the weight: ninety
cannot read the country: russia
accepted: socks, 90.0 g to Russia
1. books, 5400.0 g to Armenia
2. kettle, 1200.0 g to Kazakhstan
3. socks, 90.0 g to Russia
parcels: 3, weight: 6690.0 g, postage: 1550.0
unknown command: send everything
closing with parcels: 3, weight: 6690.0 g, postage: 1550.0
```

Task: forEach From Its Type



```
forEach :: [a] -> (a -> IO ()) -> IO ()
```

`forEach xs f` runs `f` on each element of `xs`, in order:

```
ghci>  forEach ["1. books", "2. kettle"] putStrLn
1. books
2. kettle
```

Three minutes, on paper

Write `forEach` by recursion.

Hint

What can `forEach [] f` give?
Recall `pure`.

Task: `forEach` From Its Type, Solved

```haskell
forEach :: [a] -> (a -> IO ()) -> IO ()
forEach [] _       = pure ()
forEach (x : xs) f = do
  f x
  forEach xs f
```

Task: `forEach` From Its Type, Solved

```haskell
forEach :: [a] -> (a -> IO ()) -> IO ()
forEach [] _       = pure ()
forEach (x : xs) f = do
  f x
  forEach xs f
```

The type leaves few choices. Each `a` can only go to `f`.
With no elements, there is nothing to run, so we give `pure ()`.

Task: `forEach` From Its Type, Solved



```haskell
forEach :: [a] -> (a -> IO ()) -> IO ()
forEach [] _       = pure ()
forEach (x : xs) f = do
  f x
  forEach xs f
```

The type leaves few choices. Each `a` can only go to `f`.
With no elements, there is nothing to run, so we give `pure ()`.

The library has this function twice:
as `forM_` in `Control.Monad`, and as `mapM_`, with the arguments swapped.

mapM_, forM_ and when



```
ghci> :type mapM_
mapM_ :: (Foldable t, Monad m) => (a -> m b) -> t a -> m ()
ghci> :type forM_
forM_ :: (Foldable t, Monad m) => t a -> (a -> m b) -> m ()
```

mapM_, forM_ and when

```
ghci> :type mapM_
mapM_ :: (Foldable t, Monad m) => (a -> m b) -> t a -> m ()
ghci> :type forM_
forM_ :: (Foldable t, Monad m) => t a -> (a -> m b) -> m ()
```

For lists and `IO`, choose `t = []` and `m = IO`:

```
mapM_ :: (a -> IO b) -> [a] -> IO ()
forM_ :: [a] -> (a -> IO b) -> IO ()
```

mapM_, forM_ and when



```
ghci> :type mapM_
mapM_ :: (Foldable t, Monad m) => (a -> m b) -> t a -> m ()
ghci> :type forM_
forM_ :: (Foldable t, Monad m) => t a -> (a -> m b) -> m ()
ghci> :type when
when :: Applicative f => Bool -> f () -> f ()
```

For lists and `IO`, choose `t = []` and `m = IO`:

```
mapM_ :: (a -> IO b) -> [a] -> IO ()
forM_ :: [a] -> (a -> IO b) -> IO ()
```

`when b act` runs `act` if `b` is `True`, and is `pure ()` otherwise.

`forM_` and `when` need `import Control.Monad (forM_, when)`.

Reading and Writing Files



```haskell
readFile  :: FilePath -> IO String          -- type FilePath = String
writeFile :: FilePath -> String -> IO ()
lines     :: String -> [String]             -- splits at the line breaks
unlines   :: [String] -> String             -- and back
```

Reading and Writing Files



```haskell
readFile  :: FilePath -> IO String          -- type FilePath = String
writeFile :: FilePath -> String -> IO ()
lines     :: String -> [String]             -- splits at the line breaks
unlines   :: [String] -> String             -- and back
```

```haskell
summarise :: FilePath -> FilePath -> IO ()
summarise manifest out = do
  text <- readFile manifest
  let (errs, ps) = checkManifest (lines text)
  forM_ errs putStrLn
  when (errs /= []) (putStrLn (show (length errs) ++ " lines rejected"))
  writeFile out (report ps)
  putStrLn ("report written to " ++ out)
```

Reading and Writing Files

```haskell
readFile  :: FilePath -> IO String          -- type FilePath = String
writeFile :: FilePath -> String -> IO ()
lines     :: String -> [String]             -- splits at the line breaks
unlines   :: [String] -> String             -- and back
```

```haskell
summarise :: FilePath -> FilePath -> IO ()
summarise manifest out = do
  text <- readFile manifest
  let (errs, ps) = checkManifest (lines text)
  forM_ errs putStrLn
  when (errs /= []) (putStrLn (show (length errs) ++ " lines rejected"))
  writeFile out (report ps)
  putStrLn ("report written to " ++ out)
```

```haskell
checkManifest :: [String] -> ([String], [Parcel])   -- both pure
report        :: [Parcel] -> String
```

The Whole Program

```haskell
main :: IO ()
main = do
  args <- getArgs                -- from System.Environment
  case args of
    [manifest, out] -> summarise manifest out
    [] -> do
      putStrLn "depot open: add, list, total, quit"
      clerk []
    _ -> putStrLn "usage: lecture-8 [MANIFEST REPORT]"
```

The Whole Program



```haskell
main :: IO ()
main = do
  args <- getArgs                -- from System.Environment
  case args of
    [manifest, out] -> summarise manifest out
    [] -> do
      putStrLn "depot open: add, list, total, quit"
      clerk []
    _ -> putStrLn "usage: lecture-8 [MANIFEST REPORT]"
```

`getArgs :: IO [String]` gives the words after the program's name.

The Whole Program

```haskell
main :: IO ()
main = do
  args <- getArgs                -- from System.Environment
  case args of
    [manifest, out] -> summarise manifest out
    [] -> do
      putStrLn "depot open: add, list, total, quit"
      clerk []
    _ -> putStrLn "usage: lecture-8 [MANIFEST REPORT]"
```

`getArgs :: IO [String]` gives the words after the program's name.

```
$  runghc lecture-8.hs lecture-8-manifest.txt report.txt
3. we do not carry matches
4. cannot read the weight: ninety
6. expected the contents, the weight and the country
3 lines rejected
report written to report.txt
```

`Functor` and `Monad`

The Staircase, Again

```
case checkContents c of
  Left err -> Left err
  Right c' ->
    case checkWeight w of
      Left err -> Left err
      Right w' -> Right (Parcel c' w' country)
```

Each step gives a `Left` back as it is, and passes the value in a `Right` on.

The Staircase, Again

```
case checkContents c of
  Left err -> Left err
  Right c' ->
    case checkWeight w of
      Left err -> Left err
      Right w' -> Right (Parcel c' w' country)
```

`>>=` for `Either`, in the Prelude

```
Left err >>= _ = Left err
Right x  >>= f = f x
```

Each step gives a `Left` back as it is, and passes the value in a `Right` on.

The operator `>>=`, read “bind”, does exactly this.
Its `f` is the rest of the computation:

The Staircase, Again

```
case checkContents c of
  Left err -> Left err
  Right c' ->
    case checkWeight w of
      Left err -> Left err
      Right w' -> Right (Parcel c' w' country)
```

`>>=` for `Either`, in the Prelude

```
Left err >>= _ = Left err
Right x  >>= f = f x
```

Each step gives a `Left` back as it is, and passes the value in a `Right` on.

The operator `>>=`, read “bind”, does exactly this.
Its `f` is the rest of the computation:

```
checkParcelBind c w country =
  checkContents c >>= \c' ->
  checkWeight w   >>= \w' ->
  Right (Parcel c' w' country)
```

Desugaring a `do` Block

in `Either String`

```haskell
do c' <- checkContents c
   w' <- checkWeight w
   Right (Parcel c' w' country)
```

in `IO`

```haskell
do c       <- getLine
   country <- getLine
   let text = c ++ " to " ++ country
   putStrLn ("label: " ++ text)
   putStrLn "next parcel, please"
```

[1] Simon Marlow (2010). ***Haskell 2010 Language Report.*** URL: https://www.haskell.org/onlinereport/haskell2010/, Section 3.14.

Desugaring a `do` Block

in `Either String`

```
do c' <- checkContents c
   w' <- checkWeight w
   Right (Parcel c' w' country)
```

```
checkContents c >>= \c' ->
checkWeight w   >>= \w' ->
Right (Parcel c' w' country)
```

in `IO`

```
do c       <- getLine
   country <- getLine
   let text = c ++ " to " ++ country
   putStrLn ("label: " ++ text)
   putStrLn "next parcel, please"
```

`x <- e` becomes `e >>= \x ->`, and the rest of the block goes into the lambda.

[1] Simon Marlow (2010). ***Haskell 2010 Language Report***. URL: https://www.haskell.org/onlinereport/haskell2010/, Section 3.14.

Desugaring a `do` Block



in `Either String`

```
do c' <- checkContents c
   w' <- checkWeight w
   Right (Parcel c' w' country)
```

```
checkContents c >>= \c' ->
checkWeight w   >>= \w' ->
Right (Parcel c' w' country)
```

in `IO`

```
do c       <- getLine
   country <- getLine
   let text = c ++ " to " ++ country
   putStrLn ("label: " ++ text)
   putStrLn "next parcel, please"
```

```
getLine >>= \c ->
getLine >>= \country ->
let text = c ++ " to " ++ country in
putStrLn ("label: " ++ text) >>
putStrLn "next parcel, please"
```

`x <- e` becomes `e >>= \x ->`, and the rest of the block goes into the lambda.

A `let` becomes `let ... in`, and a line `e` without `<-` becomes `e >> ...`[1].

`e >> rest` runs `e`, drops its result, and goes on with `rest`.

[1] Simon Marlow (2010). ***Haskell 2010 Language Report***. URL: https://www.haskell.org/onlinereport/haskell2010/, Section 3.14.

The Same do for Maybe



```haskell
newtype Tracking = Tracking String deriving (Show, Eq)
tracked :: [(Tracking, Parcel)]                    -- three parcels
priceOf :: Tracking -> Maybe Double
priceOf t = do
  p    <- lookup t tracked
  rate <- lookup (destination p) tariffs
  Just (postage (weight p) * rate)
```

```
ghci>  priceOf (Tracking "RA644000014RU")

ghci>  priceOf (Tracking "RA644000031RU")
```

The Same do for Maybe



```haskell
newtype Tracking = Tracking String deriving (Show, Eq)
tracked :: [(Tracking, Parcel)]                    -- three parcels
priceOf :: Tracking -> Maybe Double
priceOf t = do
  p    <- lookup t tracked
  rate <- lookup (destination p) tariffs
  Just (postage (weight p) * rate)
```

```
ghci>  priceOf (Tracking "RA644000014RU")
Just 1800.0
ghci>  priceOf (Tracking "RA644000031RU")
```

`Nothing` stops the block, and `Just` binds its value with `<-`.

The Same do for Maybe



```haskell
newtype Tracking = Tracking String deriving (Show, Eq)
tracked :: [(Tracking, Parcel)]                    -- three parcels
priceOf :: Tracking -> Maybe Double
priceOf t = do
  p    <- lookup t tracked
  rate <- lookup (destination p) tariffs
  Just (postage (weight p) * rate)
```

```
ghci> priceOf (Tracking "RA644000014RU")
Just 1800.0
ghci> priceOf (Tracking "RA644000031RU")   socks to Belarus
Nothing
```

Nothing stops the block, and **Just** binds its value with **<-**.
Belarus has no tariff, so the second `lookup` gives **Nothing**.

The Class `Monad`

```haskell
class Applicative m => Monad m where                    -- simplified
  (>>=) :: m a -> (a -> m b) -> m b
  (>>)  :: m a -> m b -> m b
```

[2] Peyton Jones and Wadler. Imperative functional programming. POPL 1993.

The Class `Monad`

```
class Applicative m => Monad m where                -- simplified
  (>>=) :: m a -> (a -> m b) -> m b
  (>>)  :: m a -> m b -> m b
```

An instance supplies these operations for a choice of `m`:
`Either String`, **`IO`**, or **`Maybe`**.

[2] Peyton Jones and Wadler. Imperative functional programming. POPL 1993.

The Class `Monad`

```
class Applicative m => Monad m where                -- simplified
  (>>=) :: m a -> (a -> m b) -> m b
  (>>)  :: m a -> m b -> m b
```

An instance supplies these operations for a choice of `m`:
`Either String`, `IO`, or `Maybe`.

With `m = Maybe`, bind has type:

```
(>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b
```

[2] Peyton Jones and Wadler. Imperative functional programming. POPL 1993.

The Class `Monad`

```
class Applicative m => Monad m where                -- simplified
  (>>=) :: m a -> (a -> m b) -> m b
  (>>)  :: m a -> m b -> m b
```

An instance supplies these operations for a choice of `m`:
`Either String`, `IO`, or `Maybe`.

With `m = Maybe`, bind has type:

```
(>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b
```

Every `Monad` instance also has `Applicative`, which supplies `pure`.

[2] Peyton Jones and Wadler. Imperative functional programming. POPL 1993.

The Class `Monad`

```
class Applicative m => Monad m where                -- simplified
  (>>=) :: m a -> (a -> m b) -> m b
  (>>)  :: m a -> m b -> m b
```

An instance supplies these operations for a choice of `m`:
`Either String`, `IO`, or `Maybe`.

With `m = Maybe`, bind has type:

```
(>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b
```

Every `Monad` instance also has `Applicative`, which supplies `pure`.

Peyton Jones and Wadler's `bindIO` is our `>>=`[2].

[2] Peyton Jones and Wadler. Imperative functional programming. POPL 1993.

`Functor`: A Pure Function on a Result



```haskell
class Functor f where
  fmap :: (a -> b) -> f a -> f b
```

```
ghci>  fmap postage (Just 1200)
Just 400.0
```

```
ghci>  fmap postage (checkWeight 50000)
Left "too heavy for a courier"
```

`Functor`: A Pure Function on a Result



```haskell
class Functor f where
  fmap :: (a -> b) -> f a -> f b
```

```
ghci> fmap postage (Just 1200)
Just 400.0
```

```
ghci> fmap postage (checkWeight 50000)
Left "too heavy for a courier"
```

`fmap` applies a pure function to a result, and keeps **`Nothing`** and **`Left`**.

On a list, `fmap` is `map`.

`Functor`: A Pure Function on a Result



```haskell
class Functor f where
  fmap :: (a -> b) -> f a -> f b
```

```
ghci> fmap postage (Just 1200)
Just 400.0
```

```
ghci> fmap postage (checkWeight 50000)
Left "too heavy for a courier"
```

`fmap` applies a pure function to a result, and keeps **`Nothing`** and **`Left`**.
On a list, `fmap` is `map`.

On an action, `fmap` applies the function to what the action gives:

```haskell
readWeight :: IO (Maybe Double)
readWeight = fmap readMaybe getLine
```

```haskell
readWeight' = do
  s <- getLine
  pure (readMaybe s)
```

`Functor`: A Pure Function on a Result



```haskell
class Functor f where
  fmap :: (a -> b) -> f a -> f b
```

```
ghci> fmap postage (Just 1200)
Just 400.0
```

```
ghci> fmap postage (checkWeight 50000)
Left "too heavy for a courier"
```

`fmap` applies a pure function to a result, and keeps **`Nothing`** and **`Left`**.
On a list, `fmap` is `map`.

On an action, `fmap` applies the function to what the action gives:

```haskell
readWeight :: IO (Maybe Double)
readWeight = fmap readMaybe getLine
```

```haskell
readWeight' = do
  s <- getLine
  pure (readMaybe s)
```

`Functor` is a superclass of **`Applicative`**, so every monad has `fmap`.

What We Can Read Now

	Racket	Haskell
print a line	`(displayln s)`, at once	`putStrLn s`, an action
a loop for the effect	`(for ([x xs]) ...)`	`forM_ xs (\x -> ...)`
a loop that keeps a value	a recursion with an accumulator	an action that calls itself
checks in a row	an error from the first that fails	a **do** block in **Either**

What We Can Read Now

	Racket	Haskell
print a line	`(displayln s)`, at once	`putStrLn s`, an action
a loop for the effect	`(for ([x xs]) ...)`	`forM_ xs (\x -> ...)`
a loop that keeps a value	a recursion with an accumulator	an action that calls itself
checks in a row	an error from the first that fails	a **do** block in **Either**

And two things we have not seen in Racket:

- effects shown in a type, as in **IO String**;
- one **do** for **IO**, **Maybe** and **Either**, through the class **Monad**.

Quick Recap

1. In `line <- getLine`, what is the type of `line`?
2. An action named by **`let`** is used on two lines of a block. How often does it run?
3. Why `readMaybe` and not `read`?
4. What does `x <- e` stand for in a **`do`** block?
5. Which class lets one **`do`** work for **`IO`**, **`Maybe`** and **`Either`**?

Quick Recap

1. In `line <- getLine`, what is the type of `line`?
   `String`. `getLine` itself is an **`IO String`**.
2. An action named by **`let`** is used on two lines of a block. How often does it run?
3. Why `readMaybe` and not `read`?
4. What does `x <- e` stand for in a **`do`** block?
5. Which class lets one **`do`** work for **`IO`**, **`Maybe`** and **`Either`**?

Quick Recap

1. In `line <- getLine`, what is the type of `line`?
   `String`. `getLine` itself is an **`IO String`**.
2. An action named by **`let`** is used on two lines of a block. How often does it run?
   Twice, since naming an action runs nothing, and each of the two lines runs it.
3. Why `readMaybe` and not `read`?
4. What does `x <- e` stand for in a **`do`** block?
5. Which class lets one **`do`** work for **`IO`**, **`Maybe`** and **`Either`**?

Quick Recap

1. In `line <- getLine`, what is the type of `line`?
   `String`. `getLine` itself is an **`IO String`**.
2. An action named by **`let`** is used on two lines of a block. How often does it run?
   Twice, since naming an action runs nothing, and each of the two lines runs it.
3. Why `readMaybe` and not `read`?
   On bad input, `read` stops the program, and `readMaybe` gives **`Nothing`**.
4. What does `x <- e` stand for in a **`do`** block?
5. Which class lets one **`do`** work for **`IO`**, **`Maybe`** and **`Either`**?

Quick Recap

1. In `line <- getLine`, what is the type of `line`?
   `String`. `getLine` itself is an **`IO String`**.
2. An action named by **`let`** is used on two lines of a block. How often does it run?
   Twice, since naming an action runs nothing, and each of the two lines runs it.
3. Why `readMaybe` and not `read`?
   On bad input, `read` stops the program, and `readMaybe` gives **`Nothing`**.
4. What does `x <- e` stand for in a **`do`** block?
   `e >>= \x -> ...`, with the rest of the block in the lambda.
5. Which class lets one **`do`** work for **`IO`**, **`Maybe`** and **`Either`**?

Quick Recap

1. In `line <- getLine`, what is the type of `line`?
   `String`. `getLine` itself is an **`IO String`**.
2. An action named by **`let`** is used on two lines of a block. How often does it run?
   Twice, since naming an action runs nothing, and each of the two lines runs it.
3. Why `readMaybe` and not `read`?
   On bad input, `read` stops the program, and `readMaybe` gives **`Nothing`**.
4. What does `x <- e` stand for in a **`do`** block?
   `e >>= \x -> ...`, with the rest of the block in the lambda.
5. Which class lets one **`do`** work for **`IO`**, **`Maybe`** and **`Either`**?
   `Monad`, through `>>=`.

Quiz for Lecture 8

- Ten minutes, on Moodle, now.
  Seven questions on this lecture, then three about the lecture itself.
- **No AI.** The quizzes are **7%** of the course between them, and the cheapest mark of the semester.

Quiz for Lecture 8

Ten minutes, on Moodle, now.
Seven questions on this lecture, then three about the lecture itself.

No AI. The quizzes are **7%** of the course between them, and the cheapest mark of the semester.

→ **The last three questions**

Say what you learned and what stayed unclear, **specifically**.
I read them before writing the next lecture.

What's Next

In today's lab you will

- write the worksheet on this lecture
- defend mini project 1 or problem set 7, if your name is drawn
- start on problem set 8

→ **Before next week: lazy evaluation**

- Read Hughes, *Why Functional Programming Matters*, Section 4 (Hughes 1989): lazy evaluation as glue.
- Load today's module in GHCi, run `main`, and type a few commands of your own. End with `quit`.

Thank you!



References

- Hughes, J. (Jan. 1989). **"Why Functional Programming Matters".** In: *The Computer Journal* 32.2, pp. 98–107. ISSN: 0010-4620. DOI: 10.1093/comjnl/32.2.98. eprint: https://academic.oup.com/comjnl/article-pdf/32/2/98/1445644/320098.pdf. URL: https://doi.org/10.1093/comjnl/32.2.98.
- Lipovača, Miran (2011). ***Learn You a Haskell for Great Good! A Beginner's Guide.*** No Starch Press. URL: https://learnyouahaskell.com/chapters (visited on 09/27/2026).
- Marlow, Simon (2010). ***Haskell 2010 Language Report.*** URL: https://www.haskell.org/onlinereport/haskell2010/ (visited on 09/27/2026).

References ii

- Peyton Jones, Simon L. and Philip Wadler (1993). **"Imperative Functional Programming".** In: *Proceedings of the 20th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL '93. Association for Computing Machinery, pp. 71–84. DOI: 10.1145/158511.158524. URL: https://doi.org/10.1145/158511.158524.
- The GHC Team (2024). ***GHC User's Guide, version 9.10.*** URL: https://downloads.haskell.org/ghc/9.10.3/docs/users_guide/ (visited on 09/27/2026).